DA6401 - Introduction to Deep Learning

Assignment-3: Implementing a Transformer for Machine Translation

General Instructions & Academic Integrity

- This is an individual assignment. Collaborations or discussions with other students are strictly prohibited.
- Tools such as ChatGPT or Claude are permitted **only** as conceptual aids or thought partners. They must not be used to generate the final code submission.
- All submissions will undergo rigorous plagiarism and AI-generated code detection. If plagiarism or unauthorized AI-assisted coding is detected, a grade of **zero** will be assigned.
- Training and test datasets must be strictly isolated. We will verify that the data split was performed properly and randomly.
- Any attempt to artificially inflate accuracy (e.g., data leakage or including test samples in the training set) will result in an immediate grade of **zero** for the entire assignment.
- You must submit a **Public W&B Report**. Ensure the link is accessible to the public during the evaluation phase; **failure to do so will result in a negative marking penalty**.
- **No extensions** will be granted beyond the provided deadline under any circumstances.
- Students are responsible for checking **Moodle** and the official course website for regular updates and clarifications regarding the assignment

Submission Details

- **Release Date:** 24th April 2026, 10:00 AM
- **Submission Deadline:** 19th May 2026, 23:59 PM
- **Late Submission Deadline:** 24th May 2026, 23:59 PM (with penalty)
- **Gradescope:** The formal submission of your code base and W&B report must be completed via **Gradescope**.

Assignment Overview

In this assignment, you will implement the landmark architecture from the paper "Attention Is All You Need" from scratch using PyTorch. Transitioning from the convolutional neural networks used in previous assignments, you will now build a purely attention-based sequence-to-sequence model. The goal is to develop a Neural Machine Translation (NMT) system capable of translating text from German to English.

- **Base Paper**: "Attention is all you need" `https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf`
- **Permitted Libraries:** `torch`, `numpy`, `matplotlib`, `scikit-learn`, `wandb`, `datasets`, `spacy`, `bleu`, `tqdm`.
- **Project Structure:** Follow the official Assignment-3 GitHub Skeleton. `https://github.com/MiRL-IITM/da6401_assignment_3`.

To manage compute resources while achieving this, we will strictly limit the assignment to the following dataset:

Multi30k Dataset: A multilingual dataset designed specifically for training and evaluating Neural Machine Translation models in a resource-constrained environment - Comprises 29,000 training pairs, 1,014 validation pairs, and 1,000 test pairs (`https://huggingface.co/datasets/bentrevett/multi30k`).

Note

- You are expected to implement the **base** architecture from the paper.
- You can adopt any tokenization scheme available in the `spacy` library. All pre-processing should done using this library only.
- Entire implementation should be in `torch`. You have to use basic building blocks in `torch` like `nn.Linear` and `nn.Module` to build the model and train it. You can implement any custom loss function or use existing loss functions in `torch`.
- For Layer Normalization use the nn.LayerNorm from `torch`.

1 Implementation & Evaluation Requirements (50 Marks)

1.1 Task 1: Scaled Dot-Product and Multi-Head Attention

Implement the Attention mechanism. You are not allowed to use `torch.nn.MultiheadAttention`.

- **Scaled Dot-Product Attention:** Implement the attention mechanism defined as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

- **Multi-Head Attention (MHA):** Implement the parallel attention heads that allow the model to jointly attend to information from different representation subspaces.
- **Masking:** Implement both padding masks (for encoder and decoder) and the look-ahead (causal) mask for the decoder to prevent positions from attending to subsequent positions.

1.2 Task 2: Transformer Encoder and Decoder Stacks

Construct the full encoder and decoder layers by following the exact sub-layer structure described in the paper.

- **Positional Encoding:** Implement the sinusoidal positional encoding to provide the model with information regarding the relative or absolute position of the tokens:

$$PE_{(pos,2i)} = \sin(pos/10000^{2i/d_{model}})$$
$$PE_{(pos,2i+1)} = \cos(pos/10000^{2i/d_{model}})$$

- **Layer Normalization & Residuals:** Implement the "Add & Norm" structure. You may choose between Pre-LayerNorm or Post-LayerNorm, but you must justify your choice in the report.

- **Point-wise Feed-Forward Networks:** Implement the two-layer linear transformation with a ReLU activation in between:

$$\text{FFN}(x) = \max(0, xW_1 + b_1)W_2 + b_2$$

1.3 Task 3: Training Pipeline and Optimization

To achieve convergence on the Multi30k dataset, you must implement specific optimization strategies mentioned in the original paper.

- **Label Smoothing:** Implement a label smoothing value of $\epsilon_{ls} = 0.1$.

- **Noam Scheduler:** Implement the learning rate schedule with a warmup phase:

$$lrate = d_{model}^{-0.5} \cdot \min(\text{step_num}^{-0.5}, \text{step_num} \cdot \text{warmup_steps}^{-1.5})$$

- **Greedy Decoding:** Write an inference function that generates translations token-by-token using the trained model.

Automated Evaluation Pipeline

The submission will be evaluated based on the following weighted criteria:

- **Multi-Head Attention [10M]:** Your `scaled_dot_product_attention` and `MultiHeadAttention` will be tested across five criteria : correctness of output shape, attention weights summing to 1 over the key dimension, masked positions receiving zero attention weight, `MultiHeadAttention` output shape under varying `d_model` and `num_heads`, and causal masking producing different outputs from unmasked attention.

- **Positional Encoding [10M]:** Your `PositionalEncoding` will be tested across five criteria : output shape preservation, even-indexed dimensions equalling $\sin(0) = 0$ at position 0, odd-indexed dimensions equalling $\cos(0) = 1$ at position 0, formula correctness at an arbitrary `(pos, dim)` pair, and the encoding being registered as a buffer rather than a trainable parameter.

- **Noam LR Scheduler [10M]:** Your `NoamScheduler` will be tested across five criteria : learning rate being monotonically increasing during warm-up, the peak occurring within 10 steps of `warmup_steps`, learning rate being monotonically decreasing after warm-up, the peak value matching the closed-form formula, and the learning rate at step 1 matching the formula.

- **Test-Set Performance [20M]:** Your best saved checkpoint will be loaded and evaluated on a held-out test set using corpus-level BLEU score via `evaluate_bleu`.

2 Weights & Biases Report (50 Marks)

You must submit a public W&B report documenting your experiments. Your report should include interactive plots, attention map visualizations, and a rigorous analysis of the following experiments.

2.1 The Necessity of the Noam Scheduler (10 Marks)

Train your model under two distinct conditions:

1. **Noam Scheduler:** Implementation of the linear warmup followed by inverse square root decay.
2. **Fixed Learning Rate:** A constant learning rate (e.g., 10^{-4}) with no warmup.

Deliverable: Overlay the training loss and validation accuracy curves. Explain why the Transformer is notoriously sensitive to the initial learning rate and how the warmup phase prevents early divergence in the self-attention layers.

2.2 Ablation: The Scaling Factor $\frac{1}{\sqrt{d_k}}$ (10 Marks)

The paper argues that for large values of d_k, the dot products grow large in magnitude, pushing the softmax function into regions where it has extremely small gradients.

- **Experiment:** Train one version with the scaling factor $\frac{1}{\sqrt{d_k}}$ and one version without it.
- **Analysis:** Log the gradient norms of the Query and Key weights during the first 1,000 steps. Relate your empirical findings to the "vanishing gradient" problem discussed in Section 3.2.1 of the paper.

2.3 Attention Rollout & Head Specialization (10 Marks)

Extract the attention weights from the last encoder layer for a single English sentence.

- **Visualization:** Log a heat map for **each individual head** in the Multi-Head Attention module.
- **Analysis:** Identify specific heads that perform distinct tasks (e.g., a head that attends to the next token, or a head that captures long-range dependencies). Do you observe "Head Redundancy"?

2.4 Positional Encoding vs. Learned Embeddings (10 Marks)

Replace the sinusoidal positional encoding with `torch.nn.Embedding` (learned positional parameters).

- **Experiment:** Compare the validation BLEU scores for both methods.
- **Theoretical Challenge:** In your report, discuss how the sinusoidal encoding allows the model to theoretically extrapolate to sequence lengths longer than those seen during training.

2.5 Decoder Sensitivity: Label Smoothing (10 Marks)

Train the model with $\epsilon_{ls} = 0.1$ and $\epsilon_{ls} = 0.0$ (standard Cross-Entropy).

- **Analysis:** Use W&B to plot the "Prediction Confidence" (softmax probability of the correct token).
- **Reflection:** Explain how label smoothing acts as a regularizer and why it prevents the model from becoming over-confident, even if it increases the overall training perplexity.